

Univerza Alma Mater Europaea

Projektna naloga pri predmetu

Odprta koda

GAMEDEALER

(Aplikacija za iskanje najboljših ponudb iger)

Avtor: Ibrahim Čaušević

Mentor: Dušan Fugina

Smer: Spletne in informacijske tehnologije

Maribor, 2026

Aplikacija GameDealer

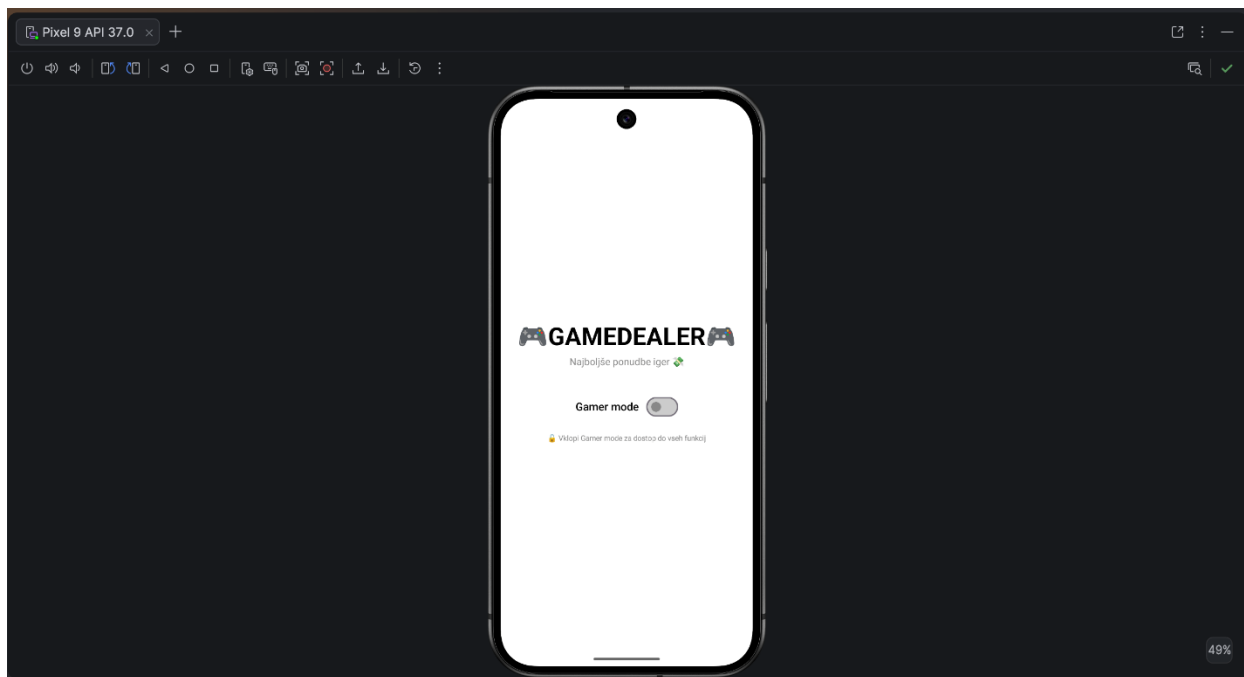
Za projektno nalogo sem se odločil razviti mobilno aplikacijo za iskanje ugodnih ponudb PC iger. Aplikacija se imenuje GameDealer in uporablja brezplačen CheapShark API, ki zbira podatke iz različnih digitalnih trgovin kot so Steam, GOG, GreenManGaming, Humble Store in druge.

Frontend

Pred začetnim zaslonom

Aplikacija se najprej prikaže z uvodnim zaslonom. To je prva stvar ki jo uporabnik vidi. Namen tega zaslona je da uporabnika pozdravi in mu pokaže osnovni namen aplikacije – iskanje najboljših ponudb iger.

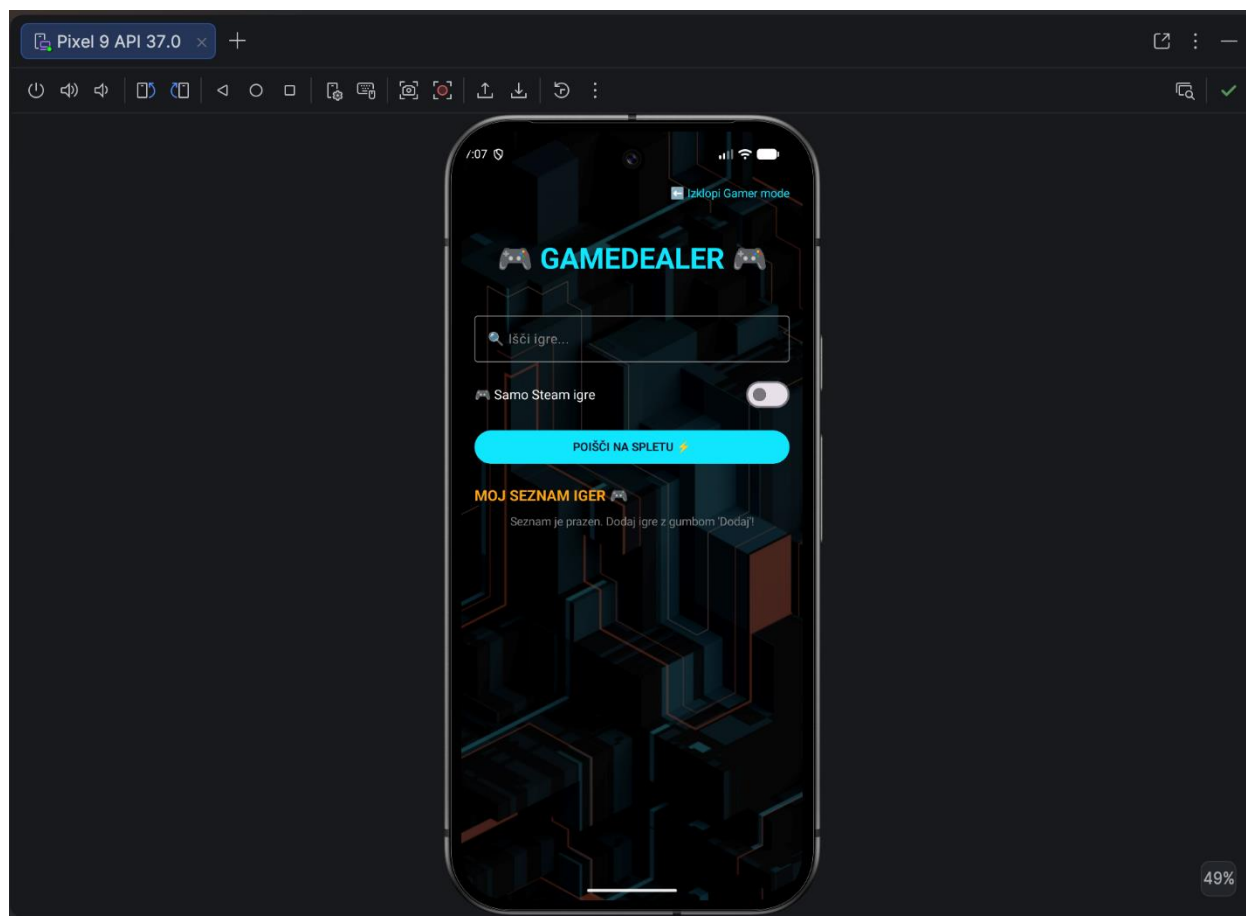
Na začetnem zaslonu je velik napis GAMEDEALER in podnaslov "Najboljše ponudbe iger". Spodaj je stikalo z napisom "Gamer mode" in pod njim pojasnilo da je potrebno vklopiti Gamer mode za dostop do vseh funkcij. Dokler uporabnik ne vklopi tega načina, nima dostopa do iskanja iger in seznama želja.



Začetni zaslon

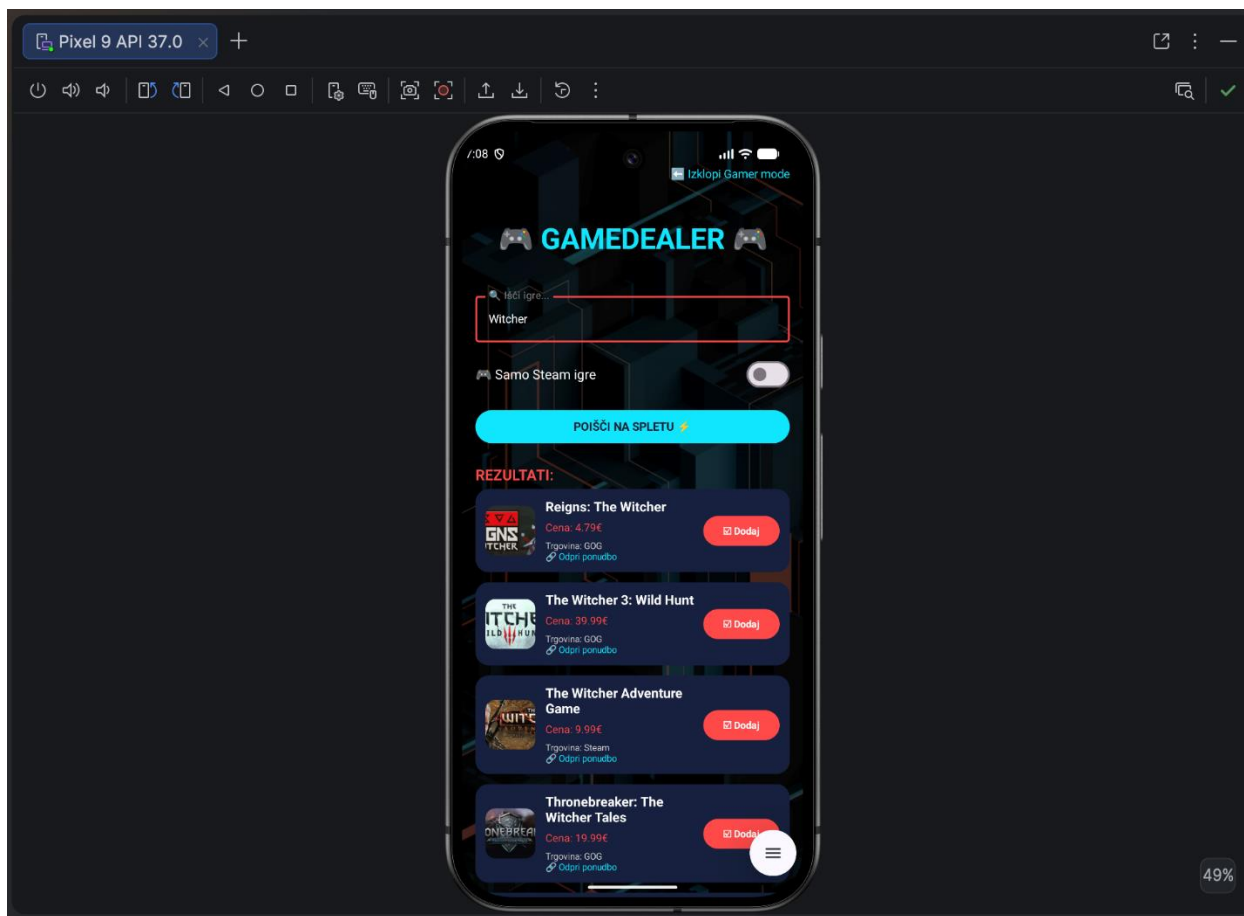
Ko je Gamer mode vklopljen, se odpre glavni vmesnik. Na vrhu je gumb za izklop Gamer mode (tako da se lahko uporabnik vrne nazaj). Sledi vnosno polje kamor uporabnik vpiše ime igre ki jo

išče. Zraven je stikalo "Samo Steam igre" in gumb "POIŠČI NA SPLETU". Spodaj je še izpis seznama iger, ki jih je uporabnik dodal na svoj seznam.



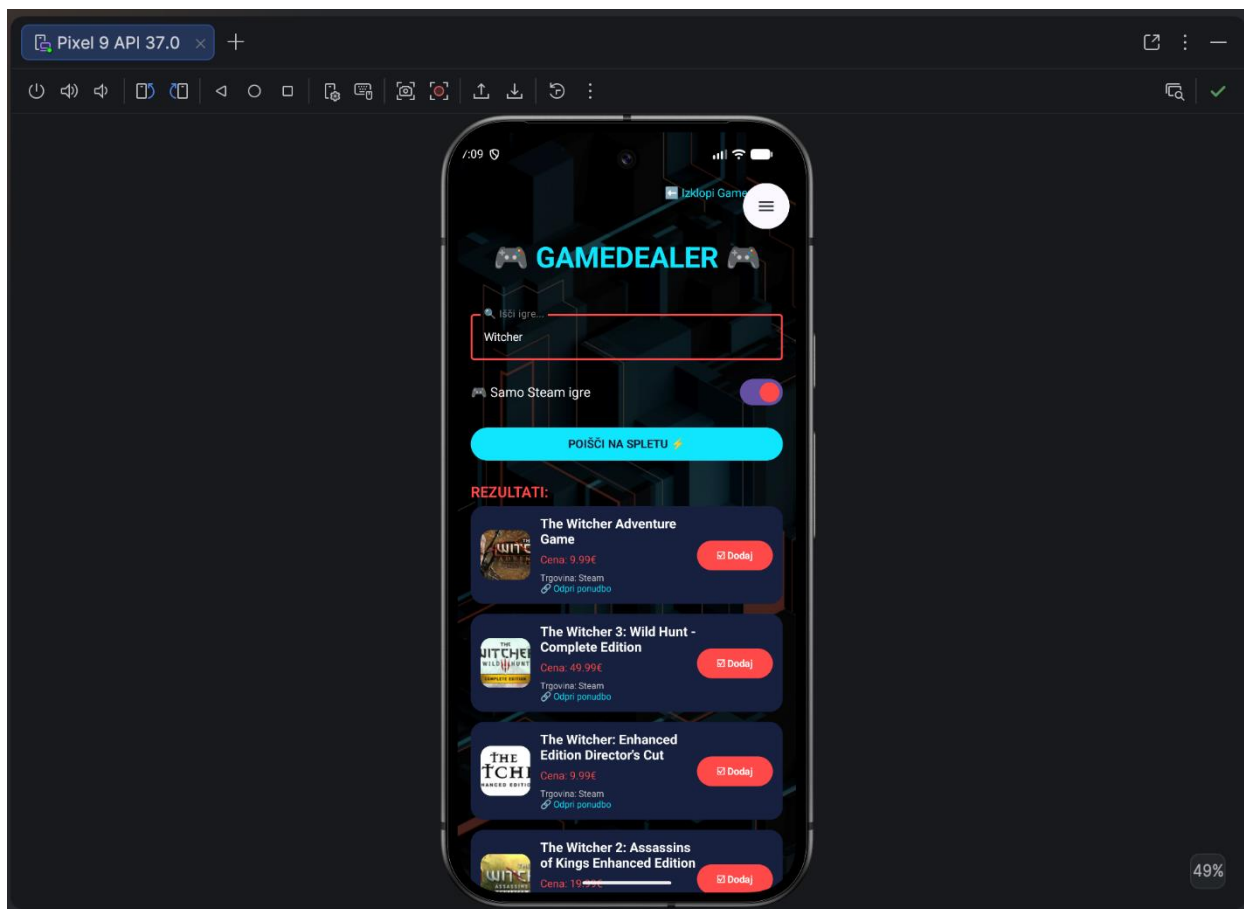
Iskanje iger

Uporabnik vpiše igro v iskalno polje in klikne gumb "POIŠČI NA SPLETU". Aplikacija poišče igre in prikaže rezultate (primer na sliki: igra Witcher). Vidimo več iger kot so "Reigns: The Witcher", "The Witcher 3: Wild Hunt" ... Vsaka igra ima sliko, svojo ceno, ime trgovine, povezavo "Odpri ponudbo" in gumb "Dodaj".



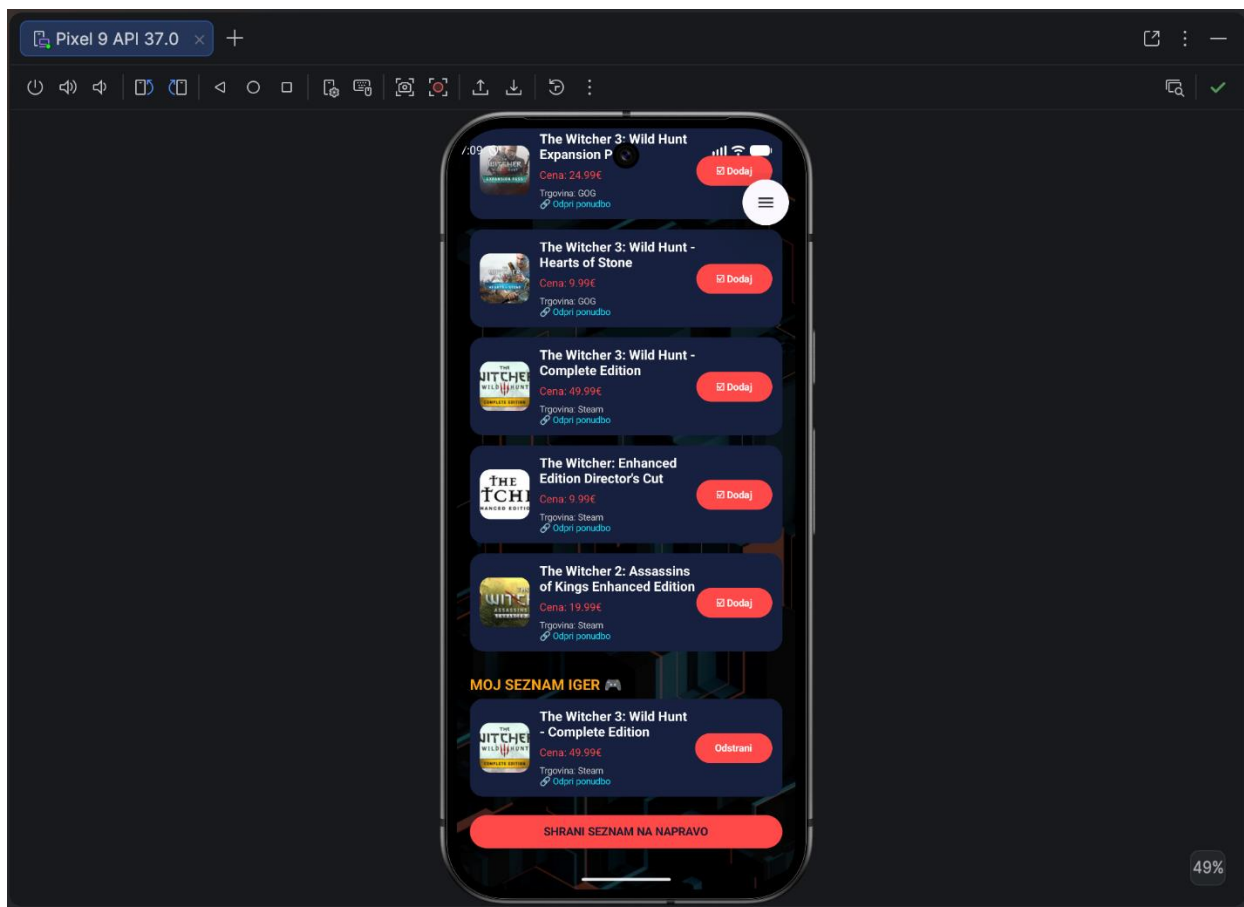
Samo Steam igre

Če uporabnik vklopi stikalo "Samo Steam igre" se prikažejo samo tiste igre, ki so na voljo v Steam trgovini. V tem primeru vidimo da so rezultati filtrirani – prikazane so samo igre s trgovino Steam, medtem ko igre iz GOG trgovine niso prikazane.



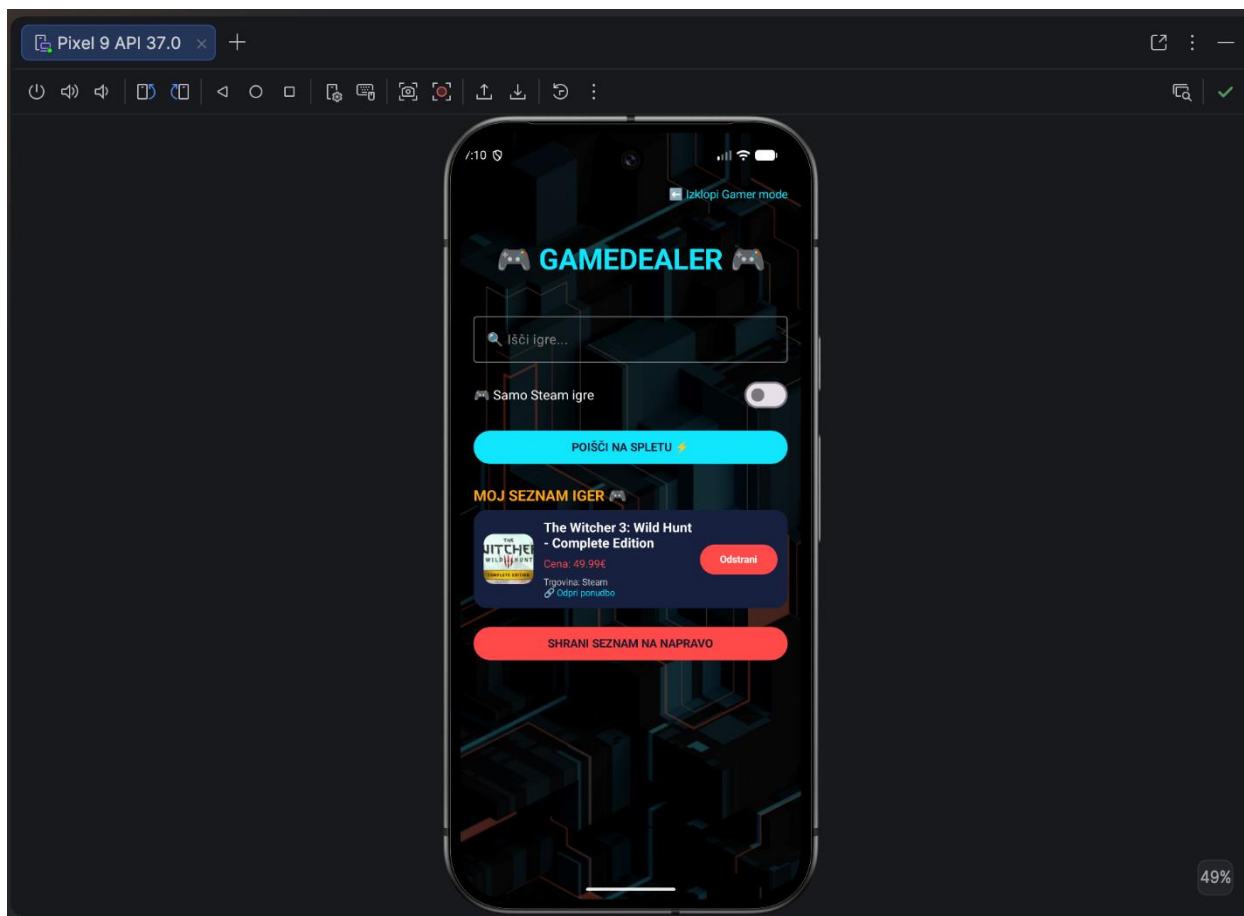
Igre, dodane na seznam

Uporabnik lahko doda ponudbo igre na svoj seznam s klikom na gumb "Dodaj". Te igre se zdaj prikazujejo v spodnjem delu "MOJ SEZNAM IGER". Vsaka igra na seznamu ima gumb "Odstrani" s katerim jo lahko uporabnik zbriše. Na dnu je gumb "SHRANI SEZNAM NA NAPRAVO" s katerim uporabnik trajno shrani svoj seznam.



Nalaganje shranjenih iger

Ko uporabnik zapre aplikacijo in jo ponovno zažene, aplikacija samodejno naloži prej shranjen seznam želja in ga prikaže.



Backend

GamingPodatki.kt

V datoteki GamingPodatki.kt so definirani vsi podatkovni razredi in vmesniki za komunikacijo s CheapShark API-jem.

Najprej so definirani podatkovni razredi, ki predstavljajo strukturo odgovorov API-ja. Razred CheapSharkIgra vsebuje attribute gameID, external, cheapest, thumb in cheapestDealID. Ta razred se uporablja za deserializacijo odgovora pri iskanju iger. Razred GameLookup vsebuje atributa info (tipa GameLookupInfo) in deals (seznam ponudb tipa Deal). Uporablja se za pridobitev podrobnih informacij o določeni igri. Razred GameLookupInfo vsebuje title, steamAppID in thumb. Razred Deal vsebuje dealID, storeID, price in retailPrice. Razred Store vsebuje storeID in storeName ter se uporablja za deserializacijo seznama trgovin.

Poleg razredov, ki jih vrača API, je definiran tudi razred Igra, ki ga aplikacija uporablja interno. Ta razred vsebuje attribute id, naslov, cijena, slikaUrl, store in dealUrl. Služi kot posredniški objekt med API-jem in uporabniškim vmesnikom.

```
MainActivity.kt  GamingPodatki.kt  Shranjevanje.kt
1 package com.example.projektnaLoga
2
3 import retrofit2.Retrofit
4 import retrofit2.converter.gson.GsonConverterFactory
5 import retrofit2.http.GET
6 import retrofit2.http.Query
7
8 1 Usage
9 data class CheapSharkIgra(
10     val gameId: String,
11     val external: String,
12     val cheapest: String,
13     val thumb: String,
14     val cheapestDealID: String? = null
15 )
16
17 1 Usage
18 data class GameLookup(
19     val info: GameLookupInfo?,
20     val deals: List<Deal>?
21 )
22
23 1 Usage
24 data class GameLookupInfo(
25     val title: String?,
26     val steamAppID: String?,
27     val thumb: String?
28 )
29
30 1 Usage
31 data class Deal(
32     val dealID: String?,
33     val storeID: String?,
34     val price: String?,
35     val retailPrice: String?
36 )
37
38 14 ^
```

Nato je definiran vmesnik CheapSharkApi, ki vsebuje tri metode z Retrofit anotacijami. Prva metoda poiščiIgre z anotacijo @GET("api/1.0/games") sprejme parametra naslov in omejitev ter vrne seznam objektov CheapSharkIgra. Druga metoda pridobiIgra prav tako uporablja endpoint api/1.0/games, vendar sprejme parameter idIgre in vrne objekt GameLookup. Tretja metoda dohvatitrgovine kliče endpoint api/1.0/stores in vrne seznam objektov Store.

```
34 data class Store(
35     val storeID: String,
36     val storeName: String
37 )
38
39 7 Usages
40 data class Igra(
41     val id: String,
42     val naslov: String,
43     val cijena: String,
44     val slikaUrl: String,
45     val store: String = "",
46     val dealUrl: String = ""
47 )
48
49 2 Usages
50 interface CheapSharkApi {
51     1 Usage
52     @GET(value = "api/1.0/games")
53     suspend fun poiščiIgre(
54         @Query(value = "title") naslov: String,
55         @Query(value = "limit") omejitev: Int = 10
56     ): List<CheapSharkIgra>
57
58     1 Usage
59     @GET(value = "api/1.0/games")
60     suspend fun pridobiIgra(@Query(value = "id") idIgre: String): GameLookup
61
62     1 Usage
63     @GET(value = "api/1.0/stores")
64     suspend fun dohvatitrgovine(): List<Store>
65 }
```


Na koncu datoteke je definiran objekt `RetrofitClient`, ki skrbi za inicializacijo Retrofit. Uporabljena je lazy inicializacija, kar pomeni da se Retrofit ustvari šele ob prvem dostopu. Nastavljen je osnovni URL `https://www.cheapshark.com/` in dodan pretvornik `GsonConverterFactory`, ki omogoča samodejno pretvorbo JSON odgovorov v Kotlin objekte.

```
3 Usages
62 object RetrofitClient {
63     val api: CheapSharkApi by lazy {
64         Retrofit.Builder()
65             .baseUrl("https://www.cheapshark.com/")
66             .addConverterFactory(GsonConverterFactory.create())
67             .build()
68             .create(CheapSharkApi::class.java)
69     }
70 }
```

Shranjevanje.kt

V datoteki `Shranjevanje.kt` sta implementirani funkciji za trajno shranjevanje in nalaganje seznama iger. Za shranjevanje je uporabljen mehanizem `SharedPreferences` v kombinaciji s knjižnico `Gson` za pretvorbo seznama v JSON niz.

Funkcija `shraniIgre` sprejme parametra `context: Context` in `igre: List<Igra>`. Znotraj funkcije se najprej pridobi `SharedPreferences` z imenom `GameDealerPrefs` v zasebnem načinu. Nato se ustvari primerek razreda `Gson`. Seznam iger se pretvori v JSON niz s klicem `gson.toJson(igre)`. Ustvari se urejevalnik `SharedPreferences` s klicem `edit()`, nato se JSON niz shrani pod ključem `MOJE_IGRE` s klicem `putString` in na koncu se spremembe potrdijo s klicem `apply()`.

Funkcija `nalozilgre` sprejme parameter `context: Context` in vrača seznam tipa `List<Igra>`. Tudi ta funkcija najprej pridobi `SharedPreferences` z istim imenom `GameDealerPrefs`. Nato se prebere JSON niz iz `SharedPreferences` s klicem `getString("MOJE_IGRE", null)`. Če je prebrani niz različen od `null`, se s pomočjo `Gson` in `TypeToken` pretvori nazaj v seznam iger in vrne. V nasprotnem primeru se vrne prazen seznam. Uporaba `TypeToken` je nujna, ker `Gson` mora vedeti, da ne pretvarja enega objekta `Igra`, ampak celoten seznam `List<Igra>`.

```

1 package com.example.projektnanaloga
2
3 import android.content.Context
4 import com.google.gson.Gson
5 import com.google.gson.reflect.TypeToken
6
7 1 Usage
8 fun shraniIgre(context: Context, igre: List<Igra>) {
9     val skupnePreference = context.getSharedPreferences( name = "GameDealerPrefs", mode = Context.MODE_PRIVATE)
10    val urednik = skupnePreference.edit()
11    val gson = Gson()
12    val json = gson.toJson( src = igre)
13    urednik.putString( key = "MOJE_IGRE", value = json)
14    urednik.apply()
15
16 1 Usage
17 fun naloziiIgre(context: Context): List<Igra> {
18     val skupnePreference = context.getSharedPreferences( name = "GameDealerPrefs", mode = Context.MODE_PRIVATE)
19     val gson = Gson()
20     val json = skupnePreference.getString( key = "MOJE_IGRE", defValue = null)
21     return if (json != null) {
22         val tip = object : TypeToken<List<Igra>>() {}.type
23         gson.fromJson(json, typeOfI = tip)
24     } else {
25         emptyList()
26     }
27 }

```

MainActivity.kt

Na začetku datoteke so uvožene knjižnice, ki jih aplikacija potrebuje. Uvoženi so Android paketi za odpiranje povezav, prikaz obvestil in delo z aktivnostjo. Uvožene so Compose knjižnice za gradnjo uporabniškega vmesnika, za pomikanje po zaslonu, za oblikovanje kartic in za upravljanje stanj. Dodatno je uvožena knjižnica Coil za nalaganje slik iz spleta.

```

1 package com.example.projektnanaloga
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.widget.Toast
7 import androidx.activity.ComponentActivity
8 import androidx.activity.compose.setContent
9 import androidx.compose.foundation.Image
10 import androidx.compose.foundation.background
11 import androidx.compose.foundation.clickable
12 import androidx.compose.foundation.layout.*
13 import androidx.compose.foundation.rememberScrollState
14 import androidx.compose.foundation.shape.RoundedCornerShape
15 import androidx.compose.foundation.verticalScroll
16 import androidx.compose.material3.*
17 import androidx.compose.runtime.*
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.draw.clip
21 import androidx.compose.ui.graphics.Color
22 import androidx.compose.ui.layout.ContentScale
23 import androidx.compose.ui.platform.LocalContext
24 import androidx.compose.ui.res.painterResource
25 import androidx.compose.ui.text.font.FontWeight
26 import androidx.compose.ui.text.style.TextAlign
27 import androidx.compose.ui.unit.dp
28 import androidx.compose.ui.unit.sp
29 import coil.compose.AsyncImage
30 import kotlinx.coroutines.launch

```

V metodi `onCreate` se nastavi Compose vsebina s klicem `setContent`. Znotraj tega se pokliče funkcija `GameDealerApp`, ki vsebuje celotno aplikacijo.

```

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(modifier = Modifier.fillMaxSize()) {
                GameDealerApp()
            }
        }
    }
}

```

GameDealerApp je glavna funkcija, ki gradi uporabniški vmesnik. Na vrhu se definira stanje gamerMode, ki pove, ali je uporabnik vklopil Gamer mode ali ne.

Če Gamer mode ni vklopljen, se prikaže začetni zaslon. Ta zaslon vsebuje naslov aplikacije, podnaslov in stikalo za vklop Gamer mode. Ko uporabnik klikne stikalo, se gamerMode spremeni v true in aplikacija preklopi na glavni vmesnik.

Ko uporabnik vklopi Gamer mode, se izvede else blok, ki vsebuje celotno logiko aplikacije. Na začetku tega bloka se pridobi kontekst s LocalContext.current. Kontekst je potreben za dostop do SharedPreferences (shranjevanje) in za prikaz obvestil. Prav tako se pridobi rememberCoroutineScope, ki omogoča zagon asinhronih operacij (klicanje API-ja) znotraj Compose.

Nato je definiranih več stanj:

-vpisanoBesedilo – hrani tisto, kar uporabnik vpiše v iskalno polje. Na začetku je prazen niz.

-samoSteam – hrani informacijo, ali je vklopljen filter za Steam igre. Na začetku je false.

-rezultatiIskanja – seznam iger, ki jih vrne API. Je prazen na začetku. Uporabljen je mutableStateListOf, kar pomeni, da se ob vsaki spremembi seznama vmesnik samodejno posodobi.

-mojSeznamZelja – seznam iger, ki jih je uporabnik shranil. Prav tako je mutableStateListOf in na začetku prazen.

-nalaganje – stanje, ki pove, ali se trenutno izvaja iskanje. Uporablja se za prikaz indikatorja za nalaganje med čakanjem na odgovor API-ja.

Definirane so tudi barve, ki se uporabljajo v aplikaciji. Barve so shranjene v spremenljivke, da jih lahko večkrat uporabimo brez ponavljanja kode.

```

43 1 Usage
44 @OptIn( ...markerClass = ExperimentalMaterial3Api::class)
45 @Composable
46 fun GameDealerApp() {
47     var gamerMode by remember { mutableStateOf( value = false) }
48
49     if (!gamerMode) {
50         Box(
51             modifier = Modifier
52                 .fillMaxSize()
53                 .background(Color.White),
54             contentAlignment = Alignment.Center
55         ) {
56             Column(
57                 horizontalAlignment = Alignment.CenterHorizontally,
58                 verticalArrangement = Arrangement.Center
59             ) {
60                 Text(
61                     text = "🎮 GAME DEALER 🎮",
62                     fontSize = 40.sp,
63                     fontWeight = FontWeight.Bold,
64                     color = Color.Black,
65                     textAlign = TextAlign.Center
66                 )
67                 Spacer(modifier = Modifier.height(8.dp))
68                 Text(
69                     text = "Najboljše ponudbe iger 🎮",
70                     fontSize = 16.sp,
71                     color = Color.Gray
72                 )
73                 Spacer(modifier = Modifier.height(40.dp))
74             }
75         }
76         Row(
77             verticalAlignment = Alignment.CenterVertically,
78             horizontalArrangement = Arrangement.Center
79         ) {
80             Text(
81                 text = "Gamer mode",
82                 fontSize = 18.sp,
83                 fontWeight = FontWeight.Medium,
84                 color = Color.Black
85             )
86             Spacer(modifier = Modifier.width(12.dp))
87             Switch(
88                 checked = gamerMode,
89                 onCheckedChange = { gamerMode = it },
90                 colors = SwitchDefaults.colors(
91                     checkedThumbColor = Color(0xFFFF4A4A),
92                     checkedTrackColor = Color(0xFFFF4A4A).copy(alpha = 0.5f),
93                     uncheckedThumbColor = Color.Gray,
94                     uncheckedTrackColor = Color.LightGray
95                 )
96             )
97         }
98         Spacer(modifier = Modifier.height(20.dp))
99         Text(
100             text = "🔒 Vklonj Gamer mode za dostop do vseh funkcij",
101             fontSize = 12.sp,
102             color = Color.Gray
103         )
104     }
105     } else {
106         val context = LocalContext.current
107         val coroutineScope = rememberCoroutineScope()
108
109         var vpisanoResedilo by remember { mutableStateOf( value = "") }
110         var samoSteam by remember { mutableStateOf( value = false) }
111         val rezultatiIskanja = remember { mutableStateListOf<Igra>() }
112         val mojSeznamZeljja = remember { mutableStateListOf<Igra>() }
113         var nalaganje by remember { mutableStateOf( value = false) }
114
115         val neonModra = Color(0xFF00E5FF)
116         val neonRdeca = Color(0xFFFF4A4A)
117         val temnaSiva = Color(0xFF1A1A2E)
118         val svetlaSiva = Color(0xFF16213E)
119         val bela = Color.White
120         val siva = Color(0xFF888888)
121         val oranzna = Color(0xFFFFA500)
122
123         LaunchedEffect( key1 = Unit) {
124             val nalozeneIgre = naloziiIgre(context)
125             mojSeznamZeljja.clear()
126             mojSeznamZeljja.addAll( elements = nalozeneIgre)
127         }
128     }

```

Definirana je rezervna mapa, ki povezuje ID-je trgovin z njihovimi imeni. To je potrebno, ker API včasih ne vrne podatkov o trgovini. Funkcija vrnilmeTrgovine prejme ID trgovine in vrne njeno ime – najprej pogleda v podatke iz API-ja, nato v rezervno mapo, če ne najde niti tam, vrne privzeto ime.

Aplikacija uporablja sliko v ozadju, ki je shranjena v mapi res/drawable. Slika se postavi z Image in zavzame celoten zaslon. Ker je slika lahko preveč svetla in bi besedilo težko brali, je čez njo postavljen še en Box s črno barvo in prosojnostjo. Tako je ozadje zatemnjeno, besedilo pa je jasno vidno.

```

129 val rezervnaMapaIrgovin = mapOf(
130     "1" to "Steam", "2" to "GamersGate", "3" to "GreenManGaming",
131     "4" to "Amazon", "5" to "GameStop", "6" to "Origin",
132     "7" to "GOG", "8" to "Direct2Drive", "11" to "Humble Store",
133     "13" to "Ubisoft Store", "15" to "Fanatical", "23" to "GameBillet",
134     "24" to "Voidu", "25" to "Epic Games", "27" to "Gamesplanet",
135     "31" to "Microsoft Store", "35" to "IndieGala"
136 )
137 var mapaIrgovin by remember { mutableStateOf<Map<String, String?>>() (Value = null) }
138 fun vrniIrgovina(storeID: String): String {
139     mapaIrgovin?.get(storeID)?.let { return it }
140     return rezervnaMapaIrgovin[storeID] ?: "Irgovina #\$storeID"
141 }
142
143 Box(modifier = Modifier.fillMaxSize()) {
144     Image(
145         painter = painterResource(id = R.drawable.background),
146         contentDescription = "Ozadje",
147         modifier = Modifier.fillMaxSize(),
148         contentScale = ContentScale.Crop
149     )
150     Box(
151         modifier = Modifier
152             .fillMaxSize()
153             .background(Color.Black.copy(alpha = 0.65f))
154     )
155
156     Column(
157         modifier = Modifier
158             .fillMaxSize()
159             .verticalScroll( state = rememberScrollState())
160             .padding( @ = 16.dp),
161         horizontalAlignment = Alignment.CenterHorizontally
162     ) {
163         Spacer(modifier = Modifier.height(50.dp))
164     }

```

Na vrhu glavnega vmesnika je besedilo "Izklopi Gamer mode". Ko uporabnik klikne nanj, se gamerMode postavi na false, kar povzroči, da se aplikacija vrne na začetni zaslon. S tem uporabnik lahko kadar koli izklopi Gamer mode in se vrne na začetni zaslon.

Iskalno polje je implementirano s komponento OutlinedTextField. Njegova vrednost je povezana s stanjem vpisanoBesedilo. Ko uporabnik nekaj vpiše, se vpisanoBesedilo posodobi.

```

164
165 Text(
166     text = "Izklopi Gamer mode",
167     color = neonModra,
168     fontSize = 14.sp,
169     modifier = Modifier
170         .align(Alignment.End)
171         .clickable { gamerMode = false }
172 )
173 Spacer(modifier = Modifier.height(50.dp))
174 Text(
175     text = "GAMEDEALER",
176     fontSize = 35.sp,
177     fontWeight = FontWeight.Bold,
178     color = neonModra,
179     textAlign = TextAlign.Center
180 )
181 Spacer(modifier = Modifier.height(40.dp))
182
183 OutlinedTextField(
184     value = vpisanoBesedilo,
185     onChange = { vpisanoBesedilo = it },
186     label = { Text("Išči igre...", color = siva) },
187     singleLine = true,
188     modifier = Modifier.fillMaxWidth(),
189     colors = OutlinedTextFieldDefaults.colors(
190         focusedBorderColor = neonRdeca,
191         unfocusedBorderColor = siva,
192         focusedLabelColor = neonRdeca,
193         cursorColor = neonRdeca,
194         focusedTextColor = bela,
195         unfocusedTextColor = bela
196     )
197 )

```

Poleg iskalnega polja je stikalo z besedilom "Samo Steam igre". Ko uporabnik vklopi stikalo, se stanje samoSteam spremeni na true. Ta filter se uporabi kasneje, ko aplikacija obdeluje rezultate iskanja – prikazane bodo samo tiste igre, katerih trgovina je Steam.

```
198         Spacer(modifier = Modifier.height(15.dp))
199
200         Row(
201             modifier = Modifier.fillMaxWidth(),
202             verticalAlignment = Alignment.CenterVertically,
203             horizontalArrangement = Arrangement.SpaceBetween
204         ) {
205             Text("🔍 Samo Steam igre", color = bela, fontSize = 16.sp)
206             Switch(
207                 checked = samoSteam,
208                 onCheckedChange = { samoSteam = it },
209                 colors = SwitchDefaults.colors(checkedThumbColor = neonRdeca)
210             )
211         }
212         Spacer(modifier = Modifier.height(15.dp))
213
214         Button(
215             onClick = {
216                 if (vpisanoBesedilo.trim().isEmpty()) {
217                     nalaganje = true
218                     coroutineScope.launch {
219                         try {
220                             if (mapaTrgovin == null) {
221                                 try {
222                                     val trgovine = RetrofitClient.api.dohvatiTrgovine()
223                                     mapaTrgovin = trgovine.associate { it.storeID to it.storeName }
224                                 } catch (e: Exception) {
225                                     mapaTrgovin = emptyMap()
226                                 }
227                             }
228                         }
229                     }
230                 }
231             }
232         )
```

Ko uporabnik klikne gumb "POIŠČI NA SPLETU", se izvede več korakov.

Najprej se preveri, ali je vpisanoBesedilo prazno. Če je prazno, se prikaže obvestilo "Vpiši iskalni niz!" in iskanje se ne izvede. Če ni prazno, se stanje nalaganje postavi na true, kar povzroči, da se na gumbu prikaže krožni indikator namesto besedila.

Potem se preveri, ali je mapaTrgovin še vedno null. Če je, se pokliče API dohvatiTrgovine, ki vrne seznam vseh trgovin. Ta seznam se pretvori v mapo (ID → ime) in shrani v mapaTrgovin. Če klic ne uspe, se mapaTrgovin nastavi na prazen seznam.

Nato se pokliče API poiisciIgre z vpisanim naslovom. Ta vrne seznam zadetkov. Vsak zadetek vsebuje osnovne podatke: ID igre, naslov, najnižjo zabeleženo ceno in sliko.

Za vsak zadetek se nato pokliče API pridobiIgra, ki vrne podrobnosti o igri, vključno s seznamom trenutnih ponudb. Iz seznama ponudb se izbere najcenejša. Iz nje se pridobi ID trgovine, ki se prek funkcije vrniImeTrgovine pretvori v ime trgovine. Prav tako se pridobi povezava do ponudbe in cena.

Če je filter za Steam vklopljen in ime trgovine ni "Steam", se ta igra preskoči – ne doda se v rezultate. Sicer se ustvari nov objekt Igra z vsemi podatki in doda v seznam rezultatiIskanja.

Ko so vse igre obdelane, se preveri, ali je rezultatiIskanja prazen. Če je prazen, se prikaže obvestilo "Ni najdenih iger". Če pride do napake med klicanjem API-ja (na primer ni internetne povezave),

se prikaže obvestilo z opisom napake. Na koncu, ne glede na to ali je iskanje uspelo ali ne, se nalaganje postavi nazaj na false, da indikator izgine.

```

228 →      val zadetki = RetrofitClient.api.poisiciigre( naslov = vpisanoBesedilo.trim())
229      rezultatiIskanja.clear()
230      for (igra in zadetki) {
231          var imeTrgovine = "Neznano"
232          var urlPonudbe = ""
233          var koncnaCena = "${igra.cheapest}%€"
234          try {
235              val pogled = RetrofitClient.api.pridobiIgra( idIgre = igra.gameID)
236              val ponudbe = pogled.deals ?: emptyList()
237              if (ponudbe.isNotEmpty()) {
238                  val najcenejsa = ponudbe.minByOrNull {
239                      it.price?.toDoubleOrNull() ?: Double.MAX_VALUE
240                  }
241                  if (najcenejsa != null) {
242                      imeTrgovine = vrniImeTrgovine( storeID = najcenejsa.storeID ?: "" )
243                      urlPonudbe = "https://www.cheapshark.com/redirect?dealID=${najcenejsa.dealID}"
244                      najcenejsa.price?.let { koncnaCena = "$it€" }
245                  }
246              }
247          } catch (e: Exception) {
248              urlPonudbe = "https://www.cheapshark.com/redirect?dealID=${igra.gameID}"
249          }
250          if (samoSteam && imeTrgovine != "Steam") {
251              continue
252          }

```

Če rezultatiIskanja ni prazen, se pod gumbom za iskanje prikaže naslov "REZULTATI". Nato se s pomočjo forEach za vsako igro v seznamu pokliče funkcija KarticaIgre, ki poskrbi za grafični prikaz posamezne igre.

Funkciji KarticaIgre se poleg podatkov o igri preda tudi akcija, ki naj se zgodi ob kliku na gumb. V tem primeru je akcija: preveri, ali igra že obstaja v mojSeznamZelja. Če ne obstaja, jo doda in prikaže obvestilo "Dodano na seznam želja!". Če že obstaja, prikaže obvestilo "Igra je že v seznamu!".

```

253      rezultatiIskanja.add(
254          Igra(
255              id = igra.gameID,
256              naslov = igra.external,
257              cena = koncnaCena,
258              slikaUrl = igra.thumb,
259              store = imeTrgovine,
260              dealUrl = urlPonudbe
261          )
262      )
263      if (rezultatiIskanja.isEmpty()) {
264          Toast.makeText(context, text = "❌ Ni najdenih iger.", duration = Toast.LENGTH_SHORT).show()
265      }
266      } catch (e: Exception) {
267          Toast.makeText(context, text = "Napaka: ${e.message}", duration = Toast.LENGTH_LONG).show()
268      } finally {
269          nalaganje = false
270      }
271      }
272      } else {
273          Toast.makeText(context, text = "🔍 Vpiši iskalni niz!", duration = Toast.LENGTH_SHORT).show()
274      }
275      },
276      modifier = Modifier.fillMaxWidth(),
277      colors = ButtonDefaults.buttonColors(containerColor = neonModra)
278  ) {
279      if (nalaganje) {
280          CircularProgressIndicator(modifier = Modifier.size(20.dp), color = temnoSiva)
281      } else {
282          Text("POIŠČI NA SPLETU ✎", color = temnoSiva, fontWeight = FontWeight.Bold)
283      }

```



```

284     }
285   }
286
287   if (rezultatiIskanja.isEmpty()) {
288     Spacer(modifier = Modifier.height(24.dp))
289     Text(
290       text = "REZULTATI:",
291       color = neonRdeca,
292       fontSize = 18.sp,
293       fontWeight = FontWeight.Bold,
294       modifier = Modifier.fillMaxWidth()
295     )
296     rezultatiIskanja.forEach { igra ->
297       KarticaIgre(
298         igra = igra,
299         besediloGumba = "Dodaj",
300         barvaGumba = neonRdeca,
301         temnaKartica = svetloSiva,
302         obKliku = {
303           if (!mojSeznamZelja.any { it.id == igra.id }) {
304             mojSeznamZelja.add(igra)
305             Toast.makeText(context, text = "✓ Dodano na seznam želja!", duration = Toast.LENGTH_SHORT).show()
306           } else {
307             Toast.makeText(context, text = "⚠ Igra je že v seznamu!", duration = Toast.LENGTH_SHORT).show()
308           }
309         }
310       )
311     }
312   }
313   Spacer(modifier = Modifier.height(24.dp))
314

```

Pod rezultati iskanja je prikazan še odsek "MOJ SEZNAM IGER". Najprej se preveri, ali je mojSeznamZelja prazen. Če je prazen, se prikaže besedilo "Seznam je prazen. Dodaj igre z gumbom 'Dodaj!'", kar uporabniku pove, kaj naj stori.

Če seznam ni prazen, se za vsako igro v mojSeznamZelja ponovno pokliče KarticaIgre. Tokrat je gumb na kartici označen z "Odstrani", akcija ob kliku pa igro odstrani iz seznama in prikaže obvestilo "Odstranjeno iz seznama!".

```

315   Text(
316     text = "MOJ SEZNAM IGER 📋",
317     color = oranžna,
318     fontSize = 18.sp,
319     fontWeight = FontWeight.Bold,
320     modifier = Modifier.fillMaxWidth()
321   )
322   if (mojSeznamZelja.isEmpty()) {
323     Text(
324       text = "Seznam je prazen. Dodaj igre z gumbom 'Dodaj!'",
325       color = siva,
326       modifier = Modifier.padding(.all = 12.dp)
327     )
328   } else {
329     mojSeznamZelja.forEach { igra ->
330       KarticaIgre(
331         igra = igra,
332         besediloGumba = "Odstrani",
333         barvaGumba = neonRdeca,
334         temnaKartica = svetloSiva,
335         obKliku = {
336           mojSeznamZelja.remove(element = igra)
337           Toast.makeText(context, text = "✖ Odstranjeno iz seznama!", duration = Toast.LENGTH_SHORT).show()
338         }
339       )
340     }
341   }
342

```

Pod seznamom želja je gumb "SHRANI SEZNAM NA NAPRAVO". Ko uporabnik klikne nanj, se pokliče funkcija shraniIgre(context, mojSeznamZelja), ki shrani celoten seznam na napravo. Po uspešnem shranjevanju se prikaže obvestilo "Seznam uspešno shranjen!".

```

341   Spacer(modifier = Modifier.height(12.dp))
342   Button(
343     onClick = {
344       shraniIgre(context, igra = mojSeznamZelja)
345       Toast.makeText(context, text = "Seznam uspešno shranjen!", duration = Toast.LENGTH_SHORT).show()
346     },
347     modifier = Modifier.fillMaxWidth(),
348     colors = ButtonDefaults.buttonColors(containerColor = neonRdeca)
349   ) {
350     Text("SHRANI SEZNAM NA NAPRAVO", color = temnoSiva, fontWeight = FontWeight.Bold)
351   }
352   Spacer(modifier = Modifier.height(30.dp))
353
354 }
355
356 }
357
358

```


Funkcija `KarticaIgre` je ločena funkcija, ki jo lahko večkrat uporabimo. Njen namen je prikazati eno igro v obliki kartice. Sprejme pet parametrov: podatke o igri, besedilo za gumb, barvo gumba, barvo ozadja kartice in akcijo, ki se zgodi ob kliku na gumb.

Znotraj funkcije se najprej pridobi kontekst, ki je potreben za odpiranje spletnih povezav. Nato se zgradi kartica s pomočjo komponente `Card`. Kartica ima zaobljene robove in rahel padding.

V notranjosti kartice je vrstica (`Row`), ki vsebuje:

- Slika igre, naloženo iz spleta z `AsyncImage`. Slika je omejene velikosti in ima zaobljene robove.
- Stolpec (`Column`) z besedilnimi podatki: naslov igre, cena in ime trgovine. Ta stolpec zavzame ves preostali prostor, tako da se gumb potisne na desno.
- Če igra vsebuje povezavo do ponudbe, je dodano klikljivo besedilo "Odpri ponudbo". Ko uporabnik klikne nanj, se odpre spletni brskalnik z ustrezno ponudbo. Če odpiranje ne uspe, se prikaže obvestilo "Ne morem odpreti povezave".
- Na desni strani je gumb z besedilom, ki ga poda klicatelj. Barva gumba je prav tako podana s parametrom. Ko uporabnik klikne gumb, se izvede podana akcija.

Ta funkcija je zasnovana tako, da je univerzalna – uporablja se tako za prikaz rezultatov iskanja (gumb "Dodaj") kot za prikaz seznama želja (gumb "Odstrani").

```
359 @Composable
360 fun KarticaIgre(
361     igra: Igra,
362     besediloGumba: String,
363     barvaGumba: Color,
364     temnaKartica: Color,
365     obKliku: () -> Unit
366 ) {
367     val context = LocalContext.current
368
369     Card(
370         modifier = Modifier
371             .fillMaxWidth()
372             .padding(vertical = 6.dp),
373         shape = RoundedCornerShape(size = 16.dp),
374         colors = CardDefaults.cardColors(containerColor = temnaKartica)
375     ) {
376         Row(
377             modifier = Modifier
378                 .fillMaxWidth()
379                 .padding(all = 12.dp),
380             verticalAlignment = Alignment.CenterVertically
381         ) {
382             AsyncImage(
383                 model = igra.slikaUrl,
384                 contentDescription = "Slika igre",
385                 modifier = Modifier
386                     .size(60.dp)
387                     .clip(shape = RoundedCornerShape(size = 12.dp)),
388                 contentScale = ContentScale.Crop
389             )
390             Spacer(modifier = Modifier.width(12.dp))
391         }
392     }
393 }
```

```

391
392 Column(modifier = Modifier.weight(1f)) {
393     Text(igra.naslov, color = Color.White, fontSize = 16.sp, fontWeight = FontWeight.Bold)
394     Spacer(modifier = Modifier.height(8.dp))
395     Text("Cena: ${igra.cena}", color = Color(0xFFFF4A4A), fontSize = 13.sp)
396     Spacer(modifier = Modifier.height(8.dp))
397     Text("Trgovina: ${igra.store}", color = Color.LightGray, fontSize = 11.sp)
398     if (igra.dealUrl.isNotEmpty()) {
399         Text(
400             text = "🔗 Odprej ponudbo",
401             color = Color(0xFF00E5FF),
402             fontSize = 11.sp,
403             modifier = Modifier.clickable {
404                 try {
405                     val namen = Intent(action = Intent.ACTION_VIEW, Uri.parse(uriString = igra.dealUrl))
406                     context.startActivity(intent = namen)
407                 } catch (e: Exception) {
408                     Toast.makeText(context, "Ne morem odpreti povezave", duration = Toast.LENGTH_SHORT).show()
409                 }
410             }
411         )
412     }
413 }
414 Button(
415     onClick = obkliku,
416     colors = ButtonDefaults.buttonColors(containerColor = barvaGumba),
417     modifier = Modifier.height(36.dp)
418 ) {
419     Text(besediloGumba, fontSize = 12.sp, fontWeight = FontWeight.Bold)
420 }
421 }
422 }
423 }

```